

Shuttle

A local coding harness backed by CortexWeave. This repository is the first development foundation for the dedicated-harness plan, using Rust and Ratatui.

Named verification plans, bounded local-model changes, explicit user review and recoverable finalization are available through the shared harness APIs, CLI, and the admitted-task terminal workspace. See [terminal qualification](#) for the interaction contract and platform limits.

Run the foundation

CortexWeave v0.5.1 is pinned to published revision [754126bfc2efd6330253826c89922148d33d9915](#), including native terminal receipts. Cargo fetches the dependency from GitHub; a sibling checkout is not required. Rust 1.98 is the tested compiler.

```
cargo run --locked -- demo
cargo run --locked -- preview
```

Inspect or direct a persisted task

```
# Create an idle scripted fixture task, then choose it from saved tasks
cargo run --locked -- task-new
cargo run --locked -- task

# Create a general-task intake and open its supervised terminal workspace.
# The profile is used only by explicit plan/edit confirmations; verification
# commands require the separate explicit host-execution approval.
cargo run --locked -- task --state-dir .shuttle\intake-01 `
  --workspace C:\dev\example --objective "Repair the failing unit tests" `
  --constraint "Preserve the public API" --plan C:\dev\example\verification-
plan.json `
  --profile .shuttle\local-model-profile.json --approve-host-execution

# Read-only durable status (use --json for automation)
cargo run --locked -- task-view --state-dir .shuttle\qwen-repair-07

# Reopen a saved task with the same explicit launch authority as needed.
cargo run --locked -- task --state-dir .shuttle\intake-01 `
  --profile .shuttle\local-model-profile.json --approve-host-execution
```

For a general task, the workspace displays only durable state and requires the same shortcut twice before every transition: Ctrl+P preflight, Ctrl+A admit, Ctrl+L plan, Ctrl+W grant, Ctrl+E edit, Ctrl+V verify, Ctrl+O offer, Ctrl+Y accept, Ctrl+N reject, Ctrl+F resume finalization, and Ctrl+D re-admit. Each operation restores the terminal and independently revalidates its durable inputs; the screen is never itself authority to write, accept,

or finalize. Ctrl+R shows the saved offer review. A model profile and host-execution approval are explicit launch-time inputs, not terminal text or ambient authority.

For a newly created scripted fixture, Ctrl+G reviews its scope and a second Ctrl+G starts/resumes the shared fixture workflow. Other saved tasks cannot be converted into scripted runs.

The demo creates an isolated fixture under `.shuttle/demo`, observes a failing content check, edits 41 to 42, rechecks it, and persists factual results in a separate CortexWeave database. Running it again resumes the same records without executing the actions again. It needs no inference or embedding server.

The terminal preview shows seven illustrative states: idle, streaming, tool execution, permission, diff, acceptance, and recovery. Tab cycles states; Esc exits (Ctrl+C also exits). Input supports typing, Unicode scalar backspace and Shift+Enter where reported; Ctrl+J is the newline fallback. Input is capped at 8 KiB. Linux supports bracketed paste; Windows uses the host's ordinary key-based paste, so pasted tabs/newlines retain host-specific behavior. The approval controls are visual examples.

```
cargo run --locked -- preview --state permission --export .shuttle/permission.svg
cargo run --locked -- preview --reduced-color
```

The SVG uses the actual Ratatui buffer. Automated Windows ConPTY and Docker Linux PTY checks exercise interaction and restoration. The user concluded the current [human review round](#) with provisional approval after the scrolling fix; separate platform checklist coverage is unrecorded.

What this establishes

- A reusable controller and model-provider boundary with a deterministic driver.
- One journal owner, one in-flight action, immutable results, and ordered delivery.
- Persisted intent before dispatch and a separate started marker before effects.
- Recovery that blocks replay of an action with unknown completion.
- Current input and permission checks before dispatching a prepared action.
- Atomic storage of bounded result artifacts and their outgoing native requests.
- Native session/task/episode creation and Event delivery through public services.
- A persisted 64-attempt provider budget and a restartable four-action fixture.
- Real host processes with exact scoped grants, bounded stdout/stderr, cancellation, deadlines, descendant cleanup, and a durable one-hour process-time budget.

The `process-spec` and `process` commands expose the executor without an inference server. See the [process execution contract](#) for grants, platform behavior, a runnable example, and recovery limitations.

The fixture's content check is development evidence. The demo ends at `awaiting_review`; it does not record user acceptance, close the substrate task, or claim a verified repair Experience. The state directory is dedicated to one fixture run. An unknown action requires inspection; a recovery/retry UI is a later increment.

Next implementation slice

Versioned verification, explicit acceptance/finalization and full-run accounting are implemented. Inspect budgets with `run-status`; see [run accounting](#) for request recovery and bounded stalls. Receipt-backed native terminal operations and automated terminal checks are also implemented, and CortexWeave's native receipt

patch is published and pinned. The current UI review round is complete for now. The [llama.cpp adapter](#) now records exact requests and bounded transport artifacts through the shared controller. Live Qwen read/check calls are qualified with writes disabled. Narrow real rustc/unittest [producer qualification](#) and durable native consolidation dispositions are implemented. The [qualified live repair workflow](#) connects real baseline/final checks to an isolated model-directed fixture repair and presents its exact state for user review. A live Qwen repair now reaches a fresh offer using exact byte arguments and has completed actual user acceptance and native finalization, with an explicit no-result consolidation disposition. Consult implementation status for recorded live outcomes and remaining scope.

The `task-view` command remains a read-only durable projection for automation or inspection:

```
shuttle task-view --state-dir C:\path\to\state
shuttle task-view --state-dir C:\path\to\state --json
```

It renders persisted task state without executing or accepting work. The `task` workspace is the separate, double-confirmed client for the supervised admitted task flow. See [terminal qualification](#).

For a real-task and cross-platform operator checklist, use [manual admitted-task qualification](#).

The controller bounds provider attempts, active time and repeated observations. Production context assembly, episode capacity handling, worktree sharing, and held-out evaluation are still ahead. See [implementation status](#) and [architecture decisions](#).

Development checks

```
cargo fmt --check
cargo check --locked
cargo clippy --locked --all-targets -- -D warnings
cargo test --locked
```

Source plan: revision 4 of `docs/dedicated-harness-plan.md` in CortexWeave history at `fe60a2334be774d0bdac67d4243d604788571eb4` (removed from its current checkout). The native receipt patch is published at `754126bfc2efd6330253826c89922148d33d9915`. The original baseline remains in Cargo metadata for historical reference. See [terminal qualification](#) for the Docker command and remaining platform limits.